Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Инженерный бизнес и менеджмент

КАФЕДРА Промышленная логистика (ИБМ3)

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
«Простое веб-приложение. Работа с API»
ПО КУРСУ: РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ

НА ТЕМУ:

*Веб-приложение операционного учёта креперии:
меню с технологическими картами блюд*

Выполнил студент ИБМ3-54Б _____ А.К. Рак
(Подпись, дата) (И.О. Фамилия)

Проверил _____ _____
(Подпись, дата) (И.О. Фамилия)

2026 г.

ХОД РАБОТЫ

Задача: взаимодействие с внешним API через XMLHttpRequest. Создать двухстраничное приложение по варианту: на главной странице вывести список чатов сообщества методом messages.getConversations с компонентом фильтра по параметру filter, на второй странице – выбранный чат методом messages.getConversationsById.

Сообщество и ключ доступа

Для работы с VK API [8] создано тестовое сообщество BiOS(learn). В разделе «Сообщения» сообщества включены сообщения и созданы четыре чата по теме креперии: «Закупки», «Смена», «Поставщики» и «Меню и техкарты» (рис. 1).

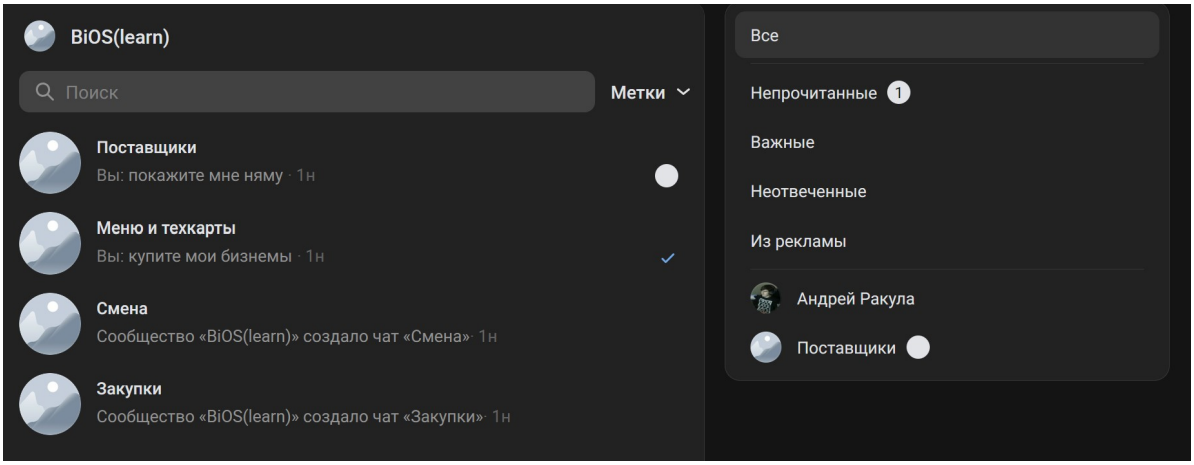


Рисунок 1 – Чаты сообщества во ВКонтакте

В разделе «Управление → Работа с API» создан ключ доступа сообщества (рис. 2). В отличие от методических указаний, где ключу выдаются все права, ключ получил только право «сообщения сообщества»: методам `messages.getConversations` и `messages.getConversationsById` с ключом сообщества нужно именно оно, а лишние права увеличивают ущерб при утечке ключа. На рисунке ключ скрыт.

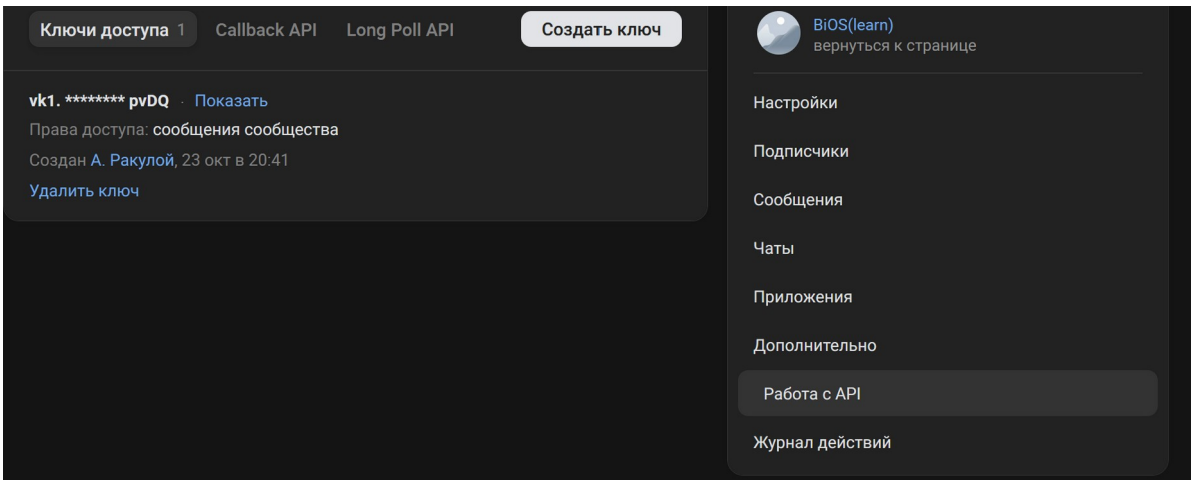


Рисунок 2 – Ключ доступа сообщества с правом на сообщения (ключ скрыт)

Константы и адреса запросов

Ключ и версия API вынесены в модуль modules/consts.js. Версия указана 5.199 – текущая по документации VK API [8] вместо устаревшей 5.131 из методических указаний. Ключ – секрет: кто его знает, может читать и отправлять сообщения от имени сообщества. Поэтому файл consts.js добавлен в .gitignore и не попадает в репозиторий, а вместо него хранится шаблон consts.example.js без ключа.

Адреса запросов формирует класс `Urls` из modules/urls.js (рис. 3). Параметры собираются объектом `URLSearchParams`, который сам кодирует специальные символы. Параметр `group_id` не передаётся: по документации [8] он нужен для сообщений сообщества только при ключе пользователя, а при ключе сообщества VK определяет сообщество по самому ключу.

```
class Urls {
    constructor() {
        this.url = 'https://api.vk.com/method'
    }

    getConversations(filter) {
        const params = new URLSearchParams({
            filter: filter,
            extended: 1,
            count: 50,
            access_token: accessToken,
            v: version
        })
        return `${this.url}/messages.getConversations?${params}`
    }

    getConversationById(peerId) {
        const params = new URLSearchParams({
            peer_ids: peerId,
            extended: 1,
            access_token: accessToken,
            v: version
        })
        return `${this.url}/messages.getConversationsById?${params}`
    }
}
```

Рисунок 3 – Класс Urls: адреса методов getConversations и getConversationsById

Запросы через XMLHttpRequest

Запросы выполняет класс `Ajax` (рис. 4). Метод `post` создаёт объект `XMLHttpRequest`, открывает POST-запрос и назначает обработчик `onreadystatechange` до вызова `send`, чтобы не пропустить смену состояния. Свойство `readyState` проходит значения от 0 до 4; обработчик ждёт значения 4 – ответ получен полностью.

Далее ответ проверяется в три этапа: статус HTTP должен быть 200, текст ответа должен разбираться как JSON, а в разобранном объекте не должно быть поля `error`. VK сообщает об ошибках (неверный ключ, нет прав) со статусом 200, кладя в ответ объект `error` вместо `response`, поэтому без третьей проверки страница падала бы молча. При ошибке вызывается колбэк `onError` с текстом для пользователя, при успехе – `onSuccess` с полем `response`.

```
class Ajax {
    post(url, onSuccess, onError) {
        const xhr = new XMLHttpRequest()
        xhr.open('POST', url)

        xhr.onreadystatechange = () => {
            if (xhr.readyState !== 4) {
                return
            }
            if (xhr.status !== 200) {
                onError(`Ошибка сети, статус ${xhr.status}`)
                return
            }
            let data
            try {
                data = JSON.parse(xhr.responseText)
            } catch (e) {
                onError('Ответ не удалось разобрать')
                return
            }
            if (data.error) {
                onError(`VK ${data.error.error_code}: ${data.error.error_msg}`)
                return
            }
            onSuccess(data.response)
        }

        xhr.send()
    }
}
```

Рисунок 4 – Класс Ajax: POST-запрос и проверка ответа

Разбор ответа VK API

Метод `messages.getConversations` возвращает объект с полями `count`, `unread_count`, `items`, а при `extended=1` – ещё `profiles` и `groups` [8]. Каждый элемент `items` содержит объект беседы `conversation` и последнее сообщение `last_message`. Название чата лежит в `conversation.chat_settings.title`, у диалога с пользователем этого поля нет и имя берётся из `profiles` по `peer.id`.

Функция `normalizeConversation` из modules/chats.js (рис. 5) превращает вложенную структуру VK в плоский объект с полями `peerId`, `title`, `photo`, `members`, `lastMessage`, `date`, `unread`, `unanswered`, `important`. Если текст последнего сообщения пуст, а в сообщении есть служебное действие `chat_create`, подставляется «Чат создан». Дата

переводится из секунд в миллисекунды и форматируется по-русски. Эту же функцию использует вторая страница.

```
export function normalizeConversation(conversation, message, profiles) {
    const isChat = conversation.peer.type === 'chat'

    let title


    if (isChat) {
        title = conversation.chat_settings.title
        members = conversation.chat_settings.members_count
        if (conversation.chat_settings.photo) {
            photo = conversation.chat_settings.photo.photo_100
        }
    } else {
        const profile = profiles.find((vkProfile) => vkProfile.id === conversation.peer.id)
        if (profile && profile.deactivated) {
            title = profile.deactivated === 'deleted' ? 'Удалённый пользователь' : 'Заблокированный пользователь'
        } else if (profile) {
            title = `${profile.first_name} ${profile.last_name}`
            photo = profile.photo_100
        } else {
            title = `Диалог ${conversation.peer.id}`
        }
    }


    let lastMessage = ''
    let date = conversation.timestamp ? new Date(conversation.timestamp * 1000).toLocaleString('ru-RU') : ''
    if (message) {
        lastMessage = message.text
        if (!lastMessage) {
            if (message.action && message.action.type === 'chat_create') {
                lastMessage = 'Чат создан'
            } else if (message.action) {
                lastMessage = 'Служебное сообщение'
            } else {
                lastMessage = 'Вложение'
            }
        }
        date = new Date(message.date * 1000).toLocaleString('ru-RU')
    }
```

Рисунок 5 – Функция normalizeConversation (фрагменты)

Главная страница и фильтр

Главная страница `KitchenChatsPage` при отрисовке выводит шапку, компонент фильтра и вызывает метод `getKitchenChats` с фильтром `all` (рис. 6). Пока идёт запрос, на странице выводится «Загрузка...», пустой результат показывается сообщением «Чатов нет», ошибка – её текстом. Поле `currentFilter` защищает от устаревших ответов: если пользователь быстро переключил фильтр, ответ на прежний запрос отбрасывается.

```
33      getKitchenChats(chatFilter) {
34          this.currentFilter = chatFilter
35          this.showStatus('Загрузка...')
36          ajax.post(urls.getConversations(chatFilter), (response) => {
37              if (!this.chatsRoot || chatFilter !== this.currentFilter) {
38                  return
39              }
40              const kitchenChats = normalizeConversations(response)
41              if (kitchenChats.length === 0) {
42                  this.chatsRoot.innerHTML = ''
43                  this.showStatus('Чатов нет')
44                  return
45              }
46              this.showStatus('')
47              this.renderKitchenChats(kitchenChats)
48          }, (message) => {
49              if (!this.chatsRoot || chatFilter !== this.currentFilter) {
50                  return
51              }
52              this.showStatus(message)
53          })
54      }
```

Рисунок 6 – Получение списка чатов в KitchenChatsPage

Компонент `ChatFilterComponent` (рис. 7) рисует четыре кнопки со значениями параметра `filter` из документации [8]: `all` – все беседы, `unread` – с непрочитанными сообщениями, `important` – отмеченные как важные, `unanswered` – без ответа (два последних значения работают только для сообщений сообщества). Нажатая кнопка передаёт значение странице, страница перерисовывает фильтр и выполняет новый запрос. Отбор выполняет сам VK по параметру запроса, а не код страницы.

```
    constructor(parent) {
        this.parent = parent
        this.chatFilters = [
            {value: 'all', label: 'Все'},
            {value: 'unread', label: 'Непрочитанные'},
            {value: 'important', label: 'Важные'},
            {value: 'unanswered', label: 'Без ответа'}
        ]
    }
```

```
    render(activeFilter, listener) {
        this.parent.innerHTML = this.getHTML(activeFilter)
        this.parent.querySelectorAll('button').forEach((button) => {
            button.addEventListener('click', () => listener(button.dataset.chatFilter))
        })
    }
```

Рисунок 7 – Компонент фильтра ChatFilterComponent

Результаты работы фильтра показаны на рисунках 8–11. Все четыре чата сообщества выводятся при фильтре «Все», при фильтре «Непрочитанные» остаётся чат «Поставщики». Отмеченных важными и неотвеченных бесед в сообществе нет, поэтому для фильтров «Важные» и «Без ответа» VK возвращает пустой список, и страница выводит «Чатов нет».

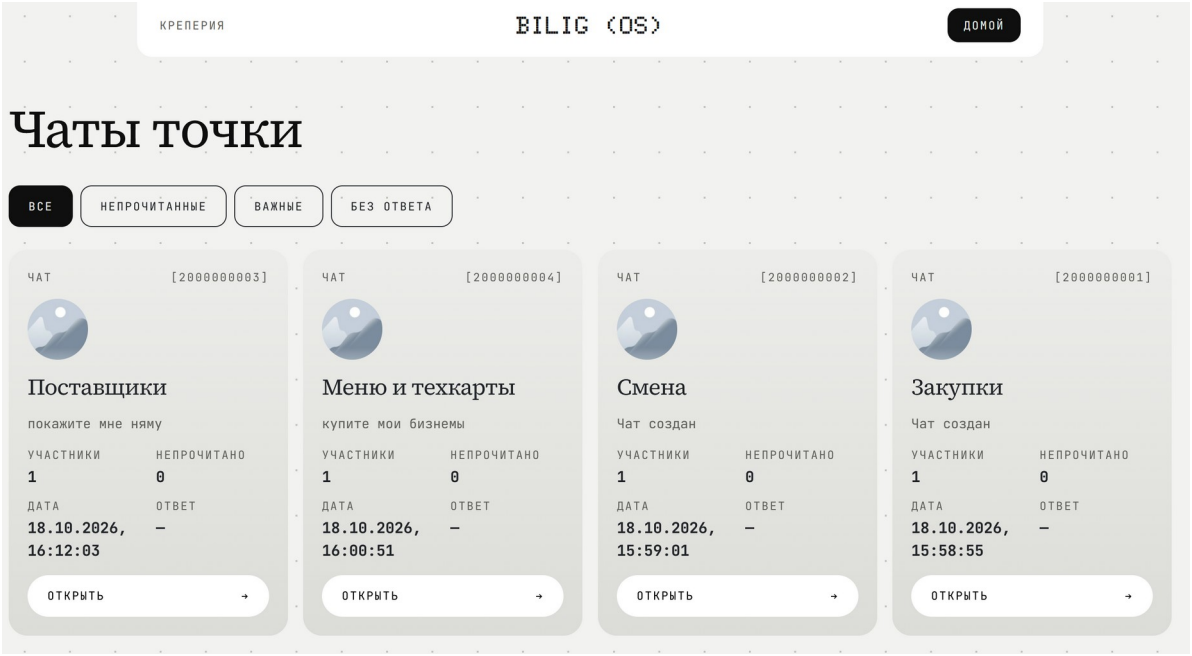


Рисунок 8 – Главная страница, фильтр «Все»

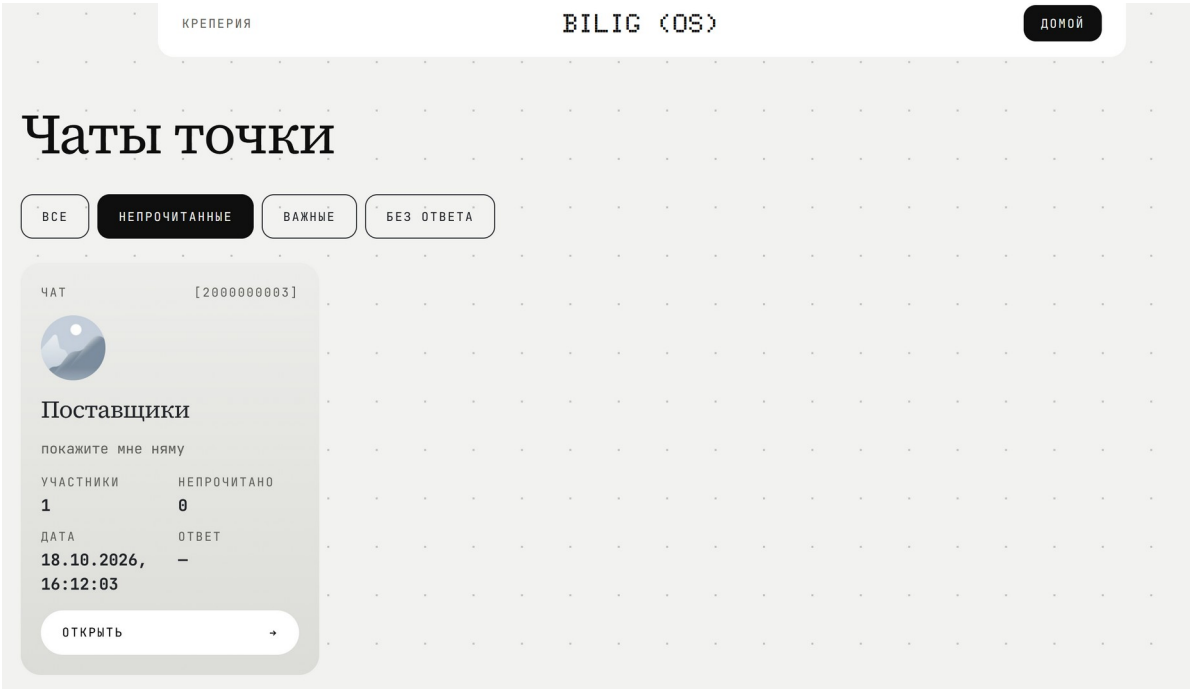


Рисунок 9 – Фильтр «Непрочитанные»



Рисунок 10 – Фильтр «Важные»: пустой результат

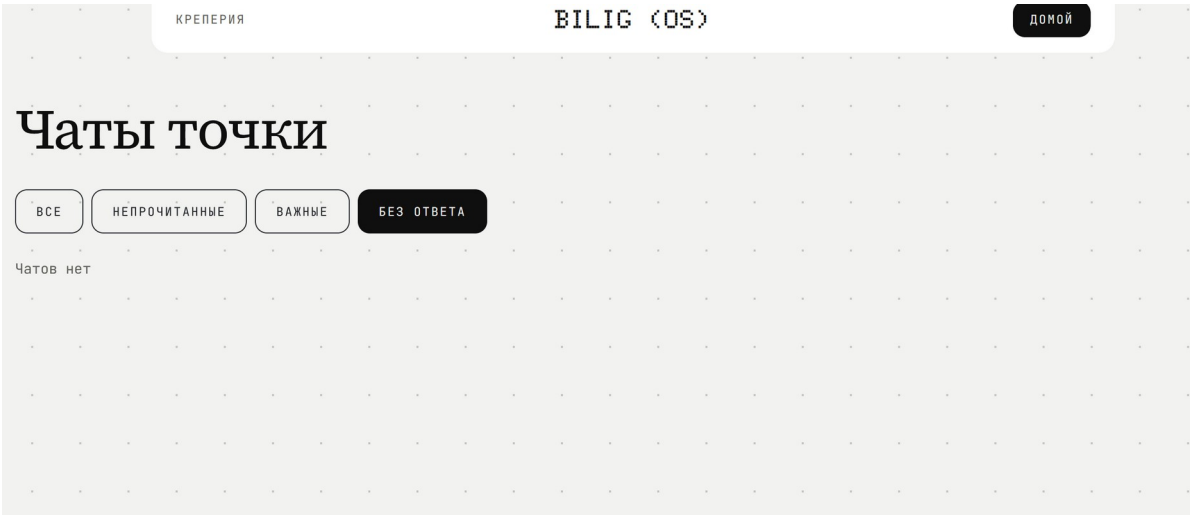


Рисунок 11 – Фильтр «Без ответа»: пустой результат

Безопасный вывод данных

Названия чатов и тексты сообщений пишут пользователи ВКонтакте, поэтому вставлять их в разметку через `innerHTML`, как в методических указаниях, небезопасно: строка вида `<img src=x onerror=...>` выполнилась бы как код (XSS-атака). Карточка чата `KitchenChatCardComponent` собирается методом `document.createElement`, а данные записываются в свойство `textContent` (рис. 12), которое браузер всегда выводит как обычный текст.

```
16      getElement(kitchenChat) {
17          const tile = document.createElement('div')
18          tile.className = 'card kitchen-chat-card'
19
20          const top = document.createElement('div')
21          top.className = 'kitchen-chat-card__top'
22          const kind = document.createElement('span')
23          kind.textContent = kitchenChat.kind
```

```
39          const title = document.createElement('h5')
40          title.className = 'card-title'
41          title.textContent = kitchenChat.title
42
43          const lastMessage = document.createElement('p')
44          lastMessage.className = 'kitchen-chat-card__message'
45          lastMessage.textContent = kitchenChat.lastMessage
```

Рисунок 12 – Сборка карточки чата через createElement и textContent

Страница выбранного чата

Кнопка «Открыть» хранит `peer_id` чата в атрибуте `data-peer-id`. Страница `KitchenChatPage` передаёт его в метод `messages.getConversationsById` параметром `peer_ids` (рис. 13). Этот метод возвращает в `items` сами объекты бесед – без обёртки `conversation` и без последнего сообщения, поэтому берётся `response.items[0]`, а не `response[0]`, как для метода `users.get` в методических указаниях. Метод

`isActual` отбрасывает ответ, если пользователь уже ушёл со страницы или открыл другой чат.

```
    isActual() {
        return this.chatRoot && this.chatRoot.dataset.peerId === this.peerId
    }

    getKitchenChat() {
        this.showStatus('Загрузка...')
        ajax.post(urls.getConversationById(this.peerId), (response) => {
            if (!this.isActual()) {
                return
            }
            if (response.items.length === 0) {
                this.showStatus('Чат не найден')
                return
            }
            this.showStatus('')
            const profiles = response.profiles || []
            this.renderKitchenChat(normalizeConversation(response.items[0], null, profiles))
        }, (message) => {
            if (!this.isActual()) {
                return
            }
            this.showStatus(message)
        })
    }
```

Рисунок 13 – Получение выбранного чата в KitchenChatPage

Страница выводит название, `peer_id`, число участников, признаки «непрочитано», «без ответа», «важный» и время последней активности (рис. 14). Для чата `peer_id` равен 2 000 000 000 плюс номер чата, поэтому у «Поставщиков» он равен 2000000003.

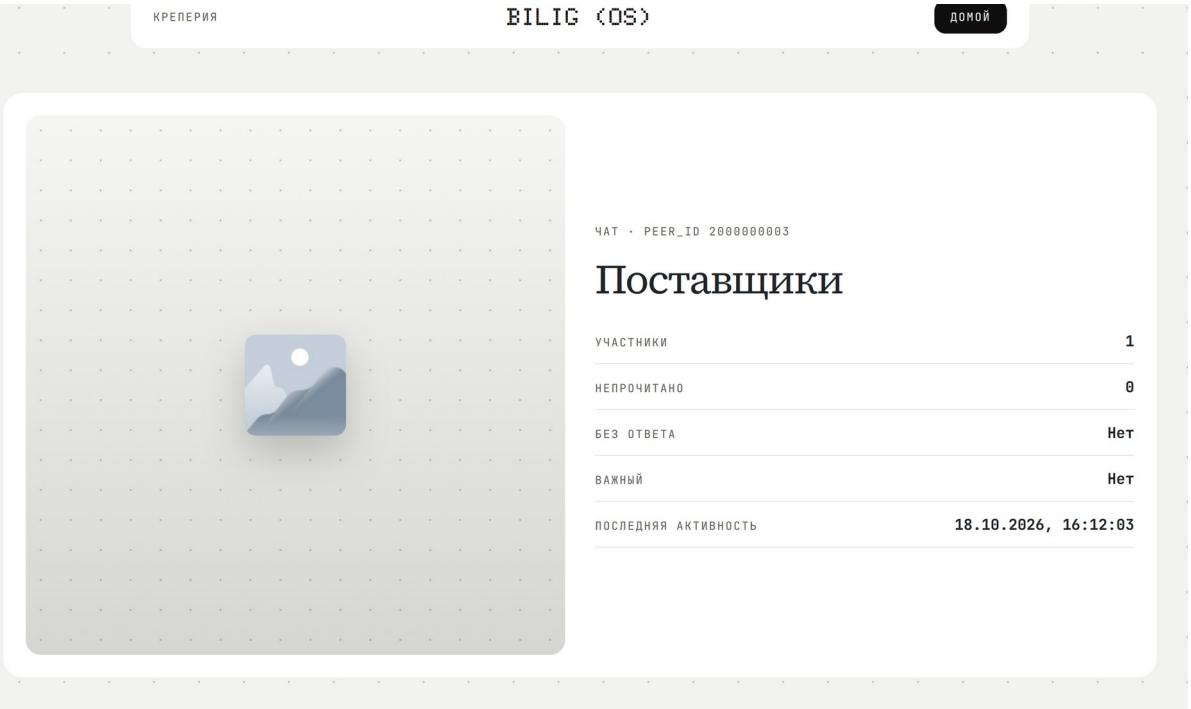


Рисунок 14 – Страница чата «Поставщики»

Ответ API во вкладке Network

Во вкладке Network инструментов разработчика виден POST-запрос к api.vk.com на метод `messages.getConversations`, отправленный из ajax.js (тип xhr). На вкладке Response показан полученный JSON: `count` – 4 беседы, `unread_count` – 1, массив `items` (рис. 15). Адрес запроса на рисунке обрезан до параметра с ключом доступа.

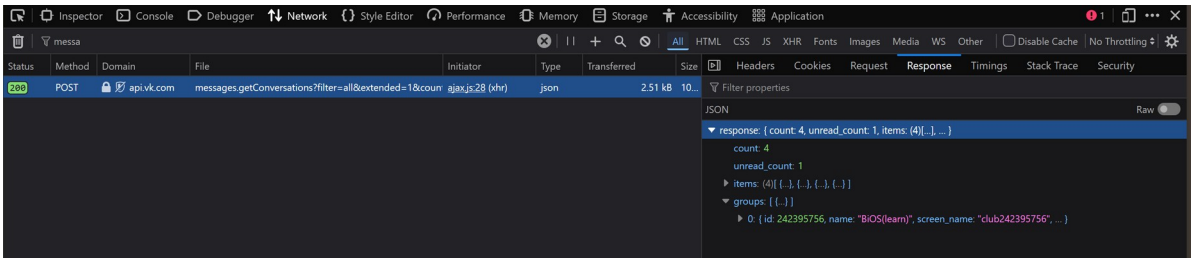


Рисунок 15 – Ответ метода messages.getConversations во вкладке Network

CORS и обработка ошибок

Браузер разрешает скрипту читать ответ с другого домена, только если сервер прислал заголовок `Access-Control-Allow-Origin`. VK API этот заголовок не отдаёт, поэтому запрос со страницы на 127.0.0.1:5500 блокируется политикой CORS: сервер отвечает, но браузер не передаёт ответ скрипту, и

`xhr.status` равен 0. Для локального запуска используется расширение браузера, снимающее это ограничение, как предлагают методические указания. В реальном приложении запросы к VK выполняет собственный сервер, на котором хранится ключ, – так ключ не попадает в браузер, а проблема CORS не возникает.

Если отключить расширение, страница не падает, а выводит «Ошибка сети, статус 0» (рис. 16). Ошибки самого VK выводятся в виде «VK код: сообщение».



Рисунок 16 – Сообщение об ошибке при блокировке запроса CORS

Исправление ошибок методических указаний

Ошибки в примерах кода методических указаний и способы их исправления приведены в таблице 1.

Таблица 1 – Исправление ошибок методических указаний

№	Ошибка в методических указаниях	Исправление
1	В ajax.js вызывается несуществующая функция getUrl(url)	Адрес передаётся напрямую: xhr.open('POST', url)
2	Обработчик onreadystatechange назначается после send()	Обработчик назначается до вызова send()
3	Ответ не проверяется: при ошибке VK возвращает объект error вместо response, и страница падает без сообщения	Проверяются статус, разбор JSON и поле error; текст ошибки выводится на странице
4	groupId = GROUP_ID – подстановка вместо числа	Константа не нужна: с ключом сообщества параметр group_id не

№	Ошибка в методических указаниях	Исправление
		передаётся
5	Версия API 5.131	Версия 5.199 по актуальной документации
6	Ключу выдаются все права	Ключ получил только право «сообщения сообщества»
7	Ключ доступа хранится в коде фронтенда	consts.js исключён из репозитория через .gitignore, в репозитории шаблон consts.example.js
8	Вторая страница берёт response[0], как для users.get	Для getConversationsById берётся response.items[0]
9	Данные вставляются в разметку через innerHTML	Элементы создаются через createElement, данные выводятся через textContent

Выполнение заданий

Выполнение задания по варианту приведено в таблице 2.

Таблица 2 – Выполнение заданий лабораторной работы № 4

№	Требование	Реализация
1	Создать сообщество, ключ доступа и несколько чатов	Сообщество BiOS(learn), четыре чата, ключ с правом на сообщения
2	Главная страница: список чатов методом messages.getConversations	KitchenChatsPage и KitchenChatCardComponent (pages/kitchen-chats, components/kitchen-chat-card)
3	Компонент фильтра по параметру filter	ChatFilterComponent: all, unread, important, unanswered (components/chat-filter)
4	Вторая страница: выбранный чат методом messages.getConversationsById	KitchenChatPage и KitchenChatComponent (pages/kitchen-chat, components/kitchen-chat)
5	Работа с API через XMLHttpRequest	Класс Ajax (modules/ajax.js)
6	Константы и адреса в отдельных модулях	modules/consts.js, modules/urls.js

№	Требование	Реализация
7	Тематические имена	kitchenChat, chatFilter, peerId, lastMessage во всех файлах проекта

ВЫВОД

В ходе работы освоено взаимодействие с внешним API через XMLHttpRequest: формирование адреса запроса с параметрами, отслеживание состояния запроса по readyState, разбор JSON-ответа и обработка ошибок сети и самого API. Приложение креперии переведено на VK API: главная страница выводит чаты сообщества методом messages.getConversations с компонентом фильтра по параметру filter, вторая страница – выбранный чат методом messages.getConversationsById. Исправлены ошибки методических указаний, данные пользователей выводятся безопасно через textContent, ключ доступа получил минимальные права и исключён из репозитория. Разобрана причина ограничения CORS и способ его обхода при локальном запуске.

Исходный код размещён в репозитории GitHub: https://github.com/rakulasi/webchi-labs, ветка lab4-creperie-vk-chats.

СПИСОК ЛИТЕРАТУРЫ

1. Курс «JavaScript»: лекции и методические указания [Электронный ресурс] // GitHub. – URL: https://github.com/iu5git/JavaScript (дата обращения: 28.10.2026).
2. MDN Web Docs. Справочник по HTML, CSS и JavaScript [Электронный ресурс]. – URL: https://developer.mozilla.org/ru/ (дата обращения: 28.10.2026).
3. Кантор И. Современный учебник JavaScript [Электронный ресурс]. – URL: https://learn.javascript.ru (дата обращения: 28.10.2026).
4. HTMLBook. Справочник по HTML и CSS [Электронный ресурс]. – URL: http://htmlbook.ru (дата обращения: 28.10.2026).
5. Google Fonts: Montserrat, Jost [Электронный ресурс]. – URL: https://fonts.google.com (дата обращения: 28.10.2026).
6. Breizh Café. Crêperie bretonne [Электронный ресурс]. – URL: https://www.breizhcafe.com (дата обращения: 28.10.2026).
7. Bootstrap 5.3. Документация [Электронный ресурс]. – URL: https://getbootstrap.com/docs/5.3/ (дата обращения: 28.10.2026).
8. ВКонтакте для разработчиков. Справочник методов API [Электронный ресурс]. – URL: https://dev.vk.com/ru/reference (дата обращения: 28.10.2026).